

CivicPulse

Technical Documentation

AI-Assisted Monsoon Civic Risk Intelligence & Response System

Challenge Track: ELCIA Smart City Drone-AI Challenge 2026

Contents

1	Introduction	2
2	AI/ML Architecture & Processing Pipeline	2
3	Input Data & Telemetry	3
3.1	DJI SRT Telemetry Parsing	4
3.2	Timestamp Association & Fallback Mechanism	4
4	Object Detection — YOLOv11m	4
4.1	Hazard Classes and Configured Thresholds	5
4.2	Detection Parsing & Spatial Moments	5
5	Object Tracking — ByteTrack	5
5.1	Two-Stage Association Formulation	5
5.2	Validation Gating & Persistence Verification	6
6	Depth Estimation — MiDaS / DPT	6
6.1	Selective Execution	6
6.2	Mathematical Representation	6
7	Severity & Risk Assessment	7
7.1	Actual Severity Formulations	7
8	Geolocation & Evidence Generation	7
8.1	Spatial Transformation	7
8.2	Evidence Artifact Pipeline	8
9	Model Evolution — YOLOv8s → YOLO11m	8
9.1	Experimental Comparison	8
9.2	Evaluation Analysis & Trade-Offs	8
10	Current Results & Limitations	10
10.1	Implementation Status	10
10.2	Software Validation	10
10.3	Current Limitations	10

1 Introduction

Urban industrial corridors experience rapid, localized infrastructure degradation during monsoon rainfall. Surface water accumulation, asphalt cavities, pavement erosion, stormwater overflow, and displaced utility covers create severe disruption and safety hazards for commuters. Traditional municipal response relies on manual ground surveys and delayed citizen grievance reports, resulting in operational latency and incomplete spatial visibility.

CivicPulse is an automated aerial computer vision and geospatial intelligence platform designed for monsoon civic risk triage. The system ingests raw drone surveillance video accompanied by synchronized SubRip (`.srt`) flight telemetry, executing a multi-stage machine learning pipeline that converts unstructured aerial footage into geolocated, deduplicated, and severity-scored municipal incident records:

Drone Video + GPS/SRT → AI/ML Processing → Hazard Detection → Tracking / Confirmation
→ Severity Assessment → Geolocation → Evidence Generation → Civic Incident

By combining pixel-level instance segmentation, multi-frame association tracking, selective relative depth profiling, and spatial database persistence, CivicPulse enforces the foundational operational rule:

$$\boxed{1 \text{ Physical Hazard} = 1 \text{ Incident}} \quad (1)$$

This technical companion report documents the models, mathematical formulations, evidence generation mechanisms, empirical evaluation metrics, and current operational boundaries implemented in the repository.

2 AI/ML Architecture & Processing Pipeline

The CivicPulse machine learning pipeline is orchestrated by the `HazardVideoPipeline` class (`src/detection/video_tracker.py`). Figure 1 illustrates the system architecture, and Figure 2 details the core video processing stages.

The pipeline executes eight sequential processing steps:

- 1. Input Ingestion & Telemetry Extraction:** Reads input drone video (`.mp4`) and parses the synchronized DJI SRT sidecar. Per-frame timestamps are computed as $t = \text{frame_idx} / \text{FPS}$ and matched to flight coordinates.
- 2. Instance Segmentation (YOLO11m-seg):** Each frame is evaluated by fine-tuned YOLO11m-seg weights, extracting bounding boxes, pixel-level polygon masks, class labels, and confidence scores.
- 3. Multi-Object Tracking (ByteTrack):** Associates detections across consecutive frames via two-stage IoU matching, assigning persistent `track_id` integers to physical hazards.
- 4. Temporal Persistence & Surface Gating:** Requires tracklets to meet per-class hit thresholds ($\text{hits} \geq \text{min_hits}$) and evaluates optical Laplacian texture variance to reject dry asphalt falsely detected as water.
- 5. Selective Depth Profiling (MiDaS / DPT-Large):** Computes relative monocular depth maps selectively on frames where depth-dependent hazards (`pothole`) are confirmed.
- 6. Severity Scoring & Priority Mapping:** Calculates a multi-factor severity score (0–100) based on surface area coverage percentage and mean relative depth drop, mapping to priority levels (P1, P2, P3).
- 7. Evidence Generation & Video Transcoding:** Writes annotated full-resolution snapshot JPEGs (with segmentation masks, bounding boxes, track IDs, and risk tags) and transcodes the complete video stream via FFmpeg (H.264 / yuv420p / `+faststart`).

8. **PostGIS Ingestion & Dashboard Dispatch:** Ingests detections, evidence file paths, and incident entities into PostgreSQL/PostGIS with spatial geometry points for live operations triage.

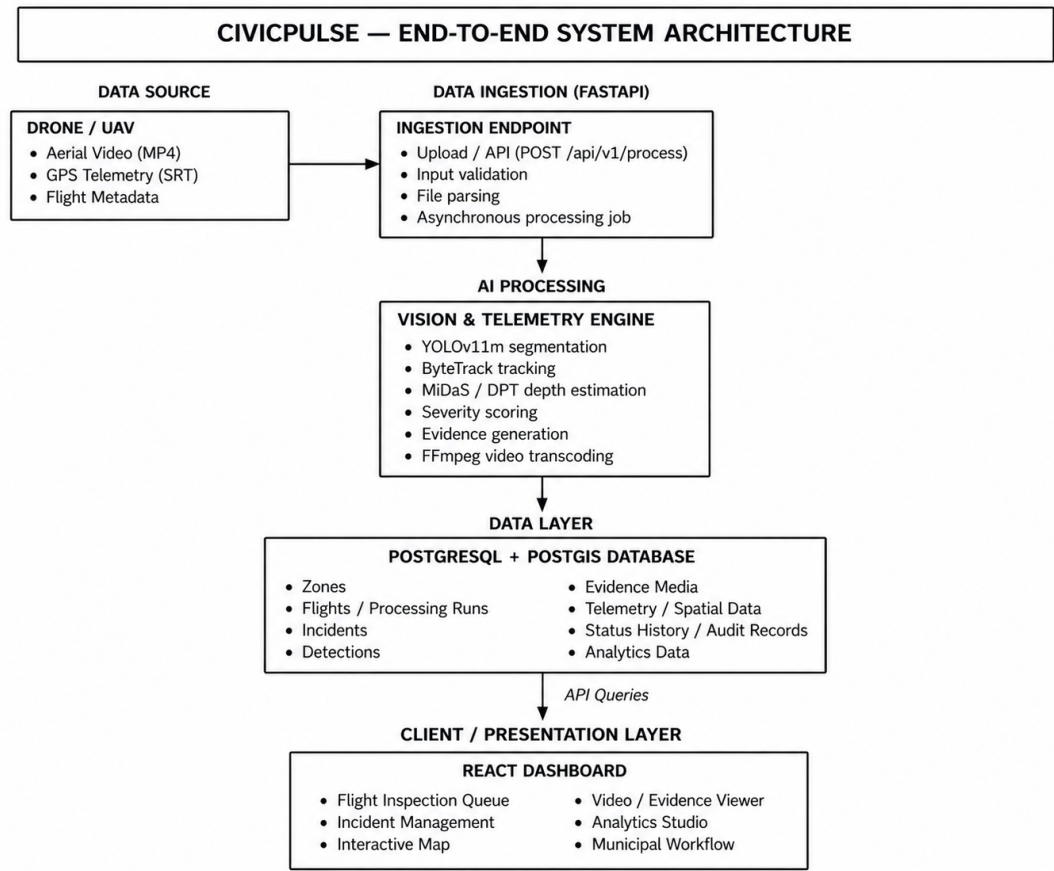


Figure 1: CivicPulse System Architecture: Drone Ingestion, AI Vision Engine, PostgreSQL/PostGIS Storage, and React Operations Dashboard.

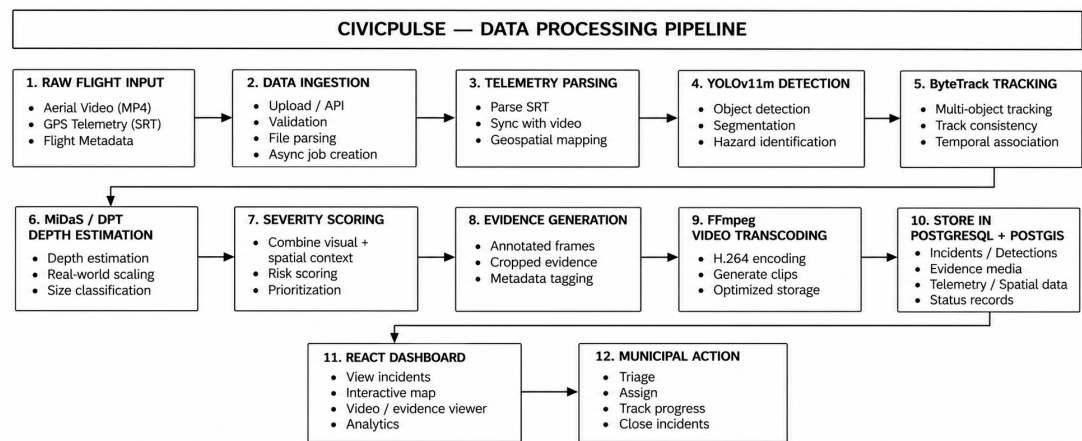


Figure 2: CivicPulse Core Video Processing Pipeline: Sequential frame evaluation, tracking, validation gates, depth profiling, and evidence export.

3 Input Data & Telemetry

CivicPulse consumes standard aerial drone surveillance streams. The video stream provides high-resolution spatial frames, while the synchronized SRT sidecar provides per-frame flight telemetry.

3.1 DJI SRT Telemetry Parsing

DJI drones record flight telemetry in SubRip subtitle sidcar files (.srt), encoding sequence index, time intervals, and bracketed or legacy coordinate tags:

```
1
00:00:00,000 --> 00:00:00,033
[latitude: 12.845214] [longitude: 77.663188] [altitude: 42.5]
```

The telemetry parser (`scripts/gps_parser.py`) extracts time-indexed GPS points using structured regular expressions supporting both modern bracketed and legacy formats:

$$\text{Pattern}_{\text{new}} = \backslash[\text{latitude}:\backslash\text{s}*(\backslash-\backslash\text{d}\backslash.+) \backslash] \backslash.*? \backslash[\text{longitude}:\backslash\text{s}*(\backslash-\backslash\text{d}\backslash.+) \backslash] \quad (2)$$

$$\text{Pattern}_{\text{old}} = \text{GPS} \backslash\text{s}*((\backslash-\backslash\text{d}\backslash.+) , \backslash\text{s}*(\backslash-\backslash\text{d}\backslash.+) \backslash) \quad (3)$$

3.2 Timestamp Association & Fallback Mechanism

For each video frame at index i and frame rate FPS, the elapsed flight time in seconds is calculated as:

$$t_i = \frac{i}{\text{FPS}} \quad (4)$$

The parser formats t_i into standard HH:MM:SS representation ($\Delta t = \text{str}(\text{timedelta}([t_i]))$) and matches it to the start timestamp of parsed SRT entries. The coordinate matching function operates as follows:

$$\text{GPS}(t_i) = \begin{cases} (\text{lat}_k, \text{lon}_k), & \text{if SRT entry } k \text{ start-time matches } \Delta t \\ (\text{lat}_{\text{last}}, \text{lon}_{\text{last}}), & \text{if } t_i > \max(\text{SRT time}) \text{ (tail hold)} \\ (\text{null}, \text{null}), & \text{if no SRT file is provided} \end{cases} \quad (5)$$

Implementation Note: Coordinate matching operates via nearest-second prefix matching of subtitle timestamps. The codebase does not perform sub-frame linear geometric spline interpolation. If no SRT telemetry is available, the ingestion service assigns the default centroid of the selected surveillance zone (e.g., EC-01). Table 1 outlines the supported input/output data schema.

Table 1: CivicPulse Input and Output Data Specifications

Field / Entity	Data Type	Operational Description
video_input	MP4 / MOV	Aerial RGB surveillance video (typically 1080p @ 30 FPS)
srt_input	Text (.srt)	Synchronized DJI telemetry sidcar with GPS coordinates
frame_idx	Integer ≥ 1	Monotonically increasing sequential frame index
timestamp_sec	Float (seconds)	Elapsed video timestamp: $t_i = \text{frame_idx}/\text{FPS}$
latitude, longitude	Float (WGS84)	GPS coordinates mapped to PostGIS Point (SRID 4326)
track_id	Integer ≥ 1	ByteTrack multi-frame object association identifier
class_name	Enum (5 classes)	Canonical hazard classification name
mask_polygon	Array of $[x, y]$	Instance segmentation contour boundary coordinates
severity_score	Integer $[0, 100]$	Computed multi-factor civic risk score
evidence_file	JPEG Image	Full annotated frame snapshot saved on hazard confirmation

4 Object Detection — YOLOv11m

CivicPulse utilizes a fine-tuned **YOLO11m-seg** instance segmentation model (`models/production/best.pt`) to detect and segment road infrastructure defects. Instance segmentation was chosen over standard bounding-box object detection because municipal road defects (such as sprawling

waterlogging patches, irregular asphalt cavities, and meandering drainage overflow streams) have irregular geometric boundaries. Bounding boxes encompass substantial healthy asphalt, distorting surface area calculations, whereas segmentation polygon masks isolate the exact defective footprint.

4.1 Hazard Classes and Configured Thresholds

The model is configured for five standardized civic hazard classes defined in `configs/config.yaml`:

1. `damaged_footpath` (Class 0): Broken walkway pavers, eroded sidewalks ($\tau_{\text{conf}} = 0.15$).
2. `drainage_overflow` (Class 1): Silted stormwater drains, roadside overflow ($\tau_{\text{conf}} = 0.30$).
3. `open_manhole` (Class 2): Missing or displaced utility/sewer access covers ($\tau_{\text{conf}} = 0.60$).
4. `pothole` (Class 3): Asphalt cavities and road surface craters ($\tau_{\text{conf}} = 0.30$).
5. `waterlogging` (Class 4): Standing rainwater inundation across road corridors ($\tau_{\text{conf}} = 0.30$).

4.2 Detection Parsing & Spatial Moments

For each predicted instance contour $C = \{(x_k, y_k)\}_{k=1}^K$, the polygon surface area in pixels is computed via Green's theorem (implemented in `cv2.contourArea`):

$$A_{\text{mask}} = \frac{1}{2} \left| \sum_{k=1}^K (x_k y_{k+1} - x_{k+1} y_k) \right| \quad (6)$$

Microscopic visual artifacts are rejected by enforcing a minimum area threshold $A_{\text{mask}} \geq 50.0$ pixels. The spatial centroid $(\bar{x}, \bar{y})$ of the polygon is calculated using spatial image moments $M_{ij} = \sum_{x,y} x^i y^j I(x, y)$:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}} \quad (7)$$

The detection output tuple passed downstream contains:

$$\mathcal{D} = \left\{ \text{track_id}, \text{class_id}, \text{class_name}, \text{confidence}, \text{bbox}, (\bar{x}, \bar{y}), C, A_{\text{mask}}, \frac{A_{\text{mask}}}{H \cdot W} \right\} \quad (8)$$

5 Object Tracking — ByteTrack

In continuous aerial video, running frame-by-frame object detection without tracking produces repetitive observations of the same physical road defect. A drone hovering over a single pothole for 5 seconds at 30 FPS would generate 150 separate detection instances.

5.1 Two-Stage Association Formulation

CivicPulse integrates ByteTrack configured in `configs/custom_bytetrack.yaml`. ByteTrack associates detections across consecutive frames by matching both high- and low-confidence boxes:

- **Stage 1 (High-Confidence Association):** Detections with confidence score $s \geq \tau_{\text{high}} = 0.20$ are matched to existing active tracks using spatial Intersection over Union (IoU) with threshold $\tau_{\text{match}} = 0.70$:

$$\text{IoU}(B_{\text{det}}, B_{\text{track}}) = \frac{\text{Area}(B_{\text{det}} \cap B_{\text{track}})}{\text{Area}(B_{\text{det}} \cup B_{\text{track}})} \geq 0.70 \quad (9)$$

- **Stage 2 (Low-Confidence Recovery):** Unmatched tracks are matched against remaining low-confidence detections ($0.08 \leq s < 0.20$) to maintain track identity through camera motion blur or partial occlusion.
- **Track Lifecycle:** Unmatched high-confidence detections ($s \geq \tau_{\text{new}} = 0.22$) initialize new tracklets. Lost tracklets are buffered for `track_buffer = 60` frames (~ 2.0 seconds at 30 FPS) before deletion.

5.2 Validation Gating & Persistence Verification

Before a tracklet is confirmed as an official incident, it must satisfy two validation gates:

1. **Temporal Persistence Gate:** The track hit counter must satisfy $\text{hits} \geq \text{min_hits}$. Standard defects require `min_hits = 2`; high-priority hazards like `open_manhole` require `min_hits = 3`; `waterlogging` requires `min_hits = 5` to reject transient puddles.
2. **Optical Laplacian Texture Variance Gate:** Fresh, dark asphalt can visually mimic standing water. For hazards with `requires_smooth_surface: true` (`waterlogging`), the region of interest (ROI) is evaluated via the variance of the Laplacian operator:

$$\sigma_{\text{Laplacian}}^2 = \text{Var}(\nabla^2 I_{\text{gray}}(\text{ROI})) = \text{Var}\left(\frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}\right) \quad (10)$$

True standing water exhibits specular, optically smooth surfaces ($\sigma^2 < 550.0$), whereas dry textured asphalt exhibits high structural grain variance ($\sigma^2 > 600.0$). Detections failing this gate are discarded.

6 Depth Estimation — MiDaS / DPT

To evaluate the vertical depression of road cavities, CivicPulse integrates zero-shot monocular depth estimation using the **DPT-Large** (Dense Prediction Transformer) architecture (`src/detection/depth_estimator.py`).

6.1 Selective Execution

Running continuous transformer depth inference on every 1080p frame would severely reduce video processing throughput. Therefore, depth estimation is executed **selectively**: it triggers exclusively on the confirmation frame of depth-dependent classes (`pothole`, configured with `needs_depth: true` in `configs/config.yaml`).

6.2 Mathematical Representation

The DPT model processes the frame image and outputs an inverse relative disparity map $d(x, y)$. This map is interpolated to native frame dimensions (H, W) via bicubic interpolation and min-max normalized to uint8 $[0, 255]$:

$$d_{\text{norm}}(x, y) = 255 \times \frac{d(x, y) - \min(d)}{\max(d) - \min(d)} \quad (11)$$

The mean relative depth score $\bar{d}_{\text{rel}} \in [0.0, 1.0]$ over the hazard's binary polygon mask M_{poly} is computed as:

$$\bar{d}_{\text{rel}} = \frac{1}{|M_{\text{poly}}|} \sum_{(x, y) \in M_{\text{poly}}} \frac{d_{\text{norm}}(x, y)}{255.0} \quad (12)$$

Operational Note: Monocular depth estimation provides **relative optical disparity** within the visual frame context. It indicates relative indentation intensity compared to surrounding road pixels and is not a metric depth measurement calibrated in physical centimeters.

7 Severity & Risk Assessment

CivicPulse calculates an automated multi-factor severity score for every confirmed hazard (`src/detection/severity_analyzer.py`).

7.1 Actual Severity Formulations

The severity score incorporates surface area coverage percentage $A_{\%}$ and mean relative depth drop $\bar{d}_{\text{rel}}$:

$$A_{\%} = \left(\frac{A_{\text{pixels}}}{H \times W} \right) \times 100 \quad (13)$$

$$S_{\text{ML}} = \min(100, \lfloor 2 \cdot A_{\%} + 50 \cdot \bar{d}_{\text{rel}} \rfloor) \quad (14)$$

For hazards where depth estimation is not required (e.g., `damaged_footpath`, `waterlogging`), $\bar{d}_{\text{rel}} = 0.0$.

For database persistence and REST API delivery (`src/services/ml_ingestion_service.py`), the integer score $S_{\text{ML}} \in [0, 100]$ is normalized to a backend scale of 0.0–10.0:

$$S_{\text{backend}} = \text{round} \left(\frac{S_{\text{ML}}}{10.0}, 2 \right) \quad (15)$$

Table 2 details the severity score thresholds, corresponding municipal priority levels, and operational triage workflows.

Table 2: CivicPulse Severity Scores, Risk Levels, and Priority Mapping

Risk Level	Condition (S_{ML})	Priority	Operational Action
CRITICAL	$S_{\text{ML}} > 70$	P1	Immediate emergency barricading; urgent pump/repair dispatch
MODERATE / HIGH	$40 < S_{\text{ML}} \leq 70$	P2	Scheduled civil maintenance within 24-hour window
LOW	$S_{\text{ML}} \leq 40$	P3	Routine monitoring; added to regular road maintenance queue

8 Geolocation & Evidence Generation

8.1 Spatial Transformation

When a hazard is confirmed at frame i (elapsed time t_i), the GPS parser matches the nearest latitude and longitude ($\text{lat}_i, \text{lon}_i$). During database ingestion, these coordinates are stored as a spatial geometry point:

$$\text{Geom} = \text{ST_SetSRID}(\text{ST_MakePoint}(\text{lon}, \text{lat}), 4326) \quad (16)$$

8.2 Evidence Artifact Pipeline

To support human verification and municipal auditing, CivicPulse generates two evidence artifacts:

1. **Annotated Evidence Snapshot:** At the confirmation frame, `draw_detections()` overlays the semi-transparent segmentation polygon ($\alpha = 0.4$), bounding box, centroid marker, ByteTrack ID, and class confidence tag.
2. **Progressive H.264 Video Transcoding:** The complete annotated video stream is transcoded via FFmpeg using the H.264 codec, yuv420p pixel format, and `+faststart` flag:

```
ffmpeg -y -i raw_stream.mp4 -c:v libx264 -preset fast -crf 22 -pix_fmt
yuv420p -movflags +faststart annotated_output.mp4
```

This enables progressive streaming in the frontend operations dashboard without requiring full-file pre-buffering.

9 Model Evolution — YOLOv8s → YOLO11m

CivicPulse evaluated two model checkpoints trained on custom 5-class civic hazard datasets:

- **v1 Baseline:** `yolov8s-seg.pt` trained at 1024×1024 resolution for 143 epochs (early-stopped with patience 30).
- **v2 Current:** `yolo11m-seg.pt` trained at 640×640 resolution for 105 epochs (early-stopped with patience 30).

9.1 Experimental Comparison

Table 3 reports the actual validation metrics recorded in the training logs (`results.csv`) of both checkpoints. The training convergence curves and normalized confusion matrix are illustrated in Figure 3, and the class-specific mask precision-recall curves are shown in Figure 4.

Table 3: Empirical Metric Comparison: YOLOv8s-seg vs. YOLO11m-seg Checkpoint Artifacts

Metric	YOLOv8s-seg (v1)	YOLO11m-seg (v2)	Observed Delta
Base Model Architecture	<code>yolov8s-seg.pt</code>	<code>yolo11m-seg.pt</code>	Upgraded C3k2/C2PSA blocks
Training Input Size	1024×1024	640×640	Standardized batching
Batch Size / Epochs	Batch 4 / 143 Epochs	Batch 16 / 105 Epochs	Faster training convergence
Box Precision (P_B)	0.710	0.674	Adjusted for recall gain
Box Recall (R_B)	0.560	0.651	+ 9.1% ($\Delta = +0.091$)
Box mAP@50	0.581	0.649	+ 6.8% ($\Delta = +0.068$)
Box mAP@50-95	0.331	0.361	+ 3.0% ($\Delta = +0.030$)
Mask Precision (P_M)	0.718	0.672	Adjusted for recall gain
Mask Recall (R_M)	0.560	0.670	+ 11.0% ($\Delta = +0.110$)
Mask mAP@50	0.578	0.646	+ 6.8% ($\Delta = +0.068$)
Mask mAP@50-95	0.333	0.345	+ 1.2% ($\Delta = +0.012$)
Validation Box Loss	1.752	1.523	− 13.1%
Validation Seg Loss	2.803	1.644	− 41.4%
Validation Cls Loss	2.354	1.280	− 45.6%

9.2 Evaluation Analysis & Trade-Offs

Under the evaluated CivicPulse training configurations, YOLO11m-seg demonstrated significant performance gains over the YOLOv8s-seg baseline:

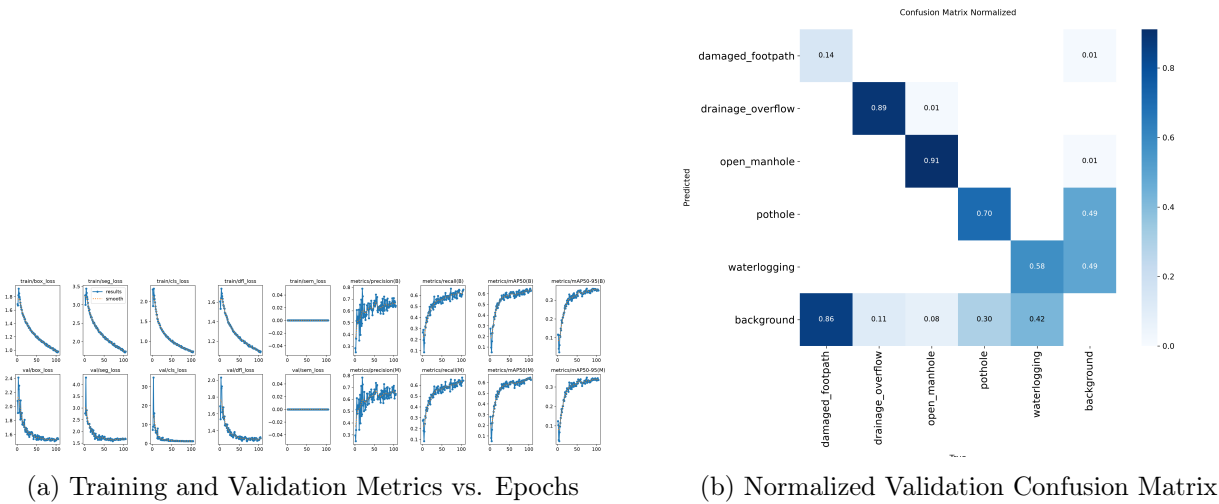


Figure 3: Experimental Training Artifacts from YOLO11m-seg Checkpoint (civimpulse_v2_yolo11m): (a) Loss and mAP metric progressions across 105 epochs, and (b) Normalized class confusion matrix showing diagonal true-positive dominance.

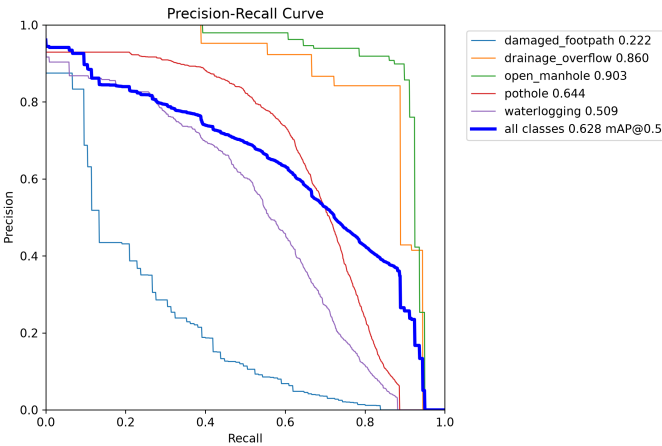


Figure 4: YOLO11m-seg Precision-Recall Curve for Mask Segmentation across 5 Hazard Classes (mAP@50 = 0.646).

1. **Significant Recall Improvement:** As shown in Table 3, mask recall increased from 56.0% to 67.0% (+11.0%), and box recall from 56.0% to 65.1% (+9.1%). In municipal hazard monitoring, higher recall reduces missed road hazards.
2. **Precision vs. Recall Trade-Off:** Precision slightly decreased (Box Precision: 0.710 $\rightarrow$ 0.674, Mask Precision: 0.718 $\rightarrow$ 0.672) as the model captured more edge cases. As seen in Figure 4, this trade-off favors overall detection coverage across diverse municipal conditions.
3. **Reduced Validation Losses:** As tracked in Figure 3a, validation classification loss dropped by 45.6% (2.354 $\rightarrow$ 1.280) and segmentation loss by 41.4% (2.803 $\rightarrow$ 1.644), demonstrating improved class discrimination and mask localization. The confusion matrix in Figure 3b confirms high separation among the five civic hazard categories.

Experimental Context: The YOLOv8s and YOLO11m runs differed in training image size (1024 vs 640), batch size (4 vs 16), and data augmentation configurations. Therefore, observed improvements reflect the combined benefits of the model upgrade, hyperparameter tuning, and dataset refinements rather than an isolated architecture change. Inference latency on uniform edge hardware was not benchmarked in these runs.

10 Current Results & Limitations

10.1 Implementation Status

- **IMPLEMENTED:** End-to-end video processing pipeline, DJI SRT telemetry parser, YOLO11m instance segmentation, ByteTrack multi-object association, selective DPT-Large depth profiling, automated multi-factor severity analyzer, annotated evidence JPEG export, H.264 video transcode, and PostgreSQL/PostGIS spatial persistence.
- **EXPERIMENTALLY EVALUATED:** YOLOv8s-seg vs. YOLO11m-seg performance across 5 hazard classes on training/validation splits with exact CSV metrics and confusion matrix artifacts.

10.2 Software Validation

Separate from machine learning metrics, the software codebase maintains rigorous automated test coverage:

- **Backend Test Suite (Pytest): 147 passing tests** covering database migrations, API routes, repository queries, spatial conversions, and evidence snapshot generation.
- **Frontend Test Suite (Vitest): 123 passing tests (13 test suites)** verifying incident queues, state machine transitions, video playback seeking, and analytics calculations.

10.3 Current Limitations

1. **Relative vs. Metric Depth:** Monocular depth estimation outputs relative inverse disparity rather than calibrated metric millimeters. It provides qualitative cavity rankings within a single frame context.
2. **Telemetry Dependency:** Spatial accuracy relies on the presence and precision of DJI SRT flight subtitles. Videos without SRT telemetry fall back to operational zone defaults.
3. **Environmental Domain Variations:** Severe downpours, low-light dusk flights, and direct sun specular glint on wet surfaces introduce visual domain shifts that can affect detection confidence.
4. **Compute Requirements:** Selective DPT-Large depth estimation and 1080p video tracking require GPU acceleration (NVIDIA CUDA or Apple Silicon MPS) for near-real-time throughput.